

Robot Movement Planning

Constraint Programming

Alessio Russo (mat. 376856)

1 Problem Statement

We consider a stream of identical boxes arriving on a belt. Each box has size $x \times y$, where x is parallel to the belt direction and y is orthogonal to it. A target pallet capable of containing n boxes is given. Each box i has a destination position on the pallet, given by $(P_x[i], P_y[i])$, where $i = 1, \dots, n$. The robot can pick a consecutive set of boxes from the stream. Since boxes on the belt have no gaps between them, each picked set is contiguous along the x -axis. Given a gripper size S , the total length of the picked set must be at most $S + x$. This corresponds to an overflow of at most $x/2$ on each side of the gripper. Therefore, the maximum number of boxes that can be picked at once is $\left\lfloor \frac{S+x}{x} \right\rfloor$. After picking a set of boxes, the robot releases it on the pallet.

The gripper has two parallel plates, but only one plate can move during release. For this reason, the opening side must be free. It can face either the outside of the pallet or a position that has not been filled yet. We split the problem into two parts:

1. **Chunking.** Identify the sets of boxes that can be picked together. Each chunk must contain boxes on the same row, aligned along the x -axis, and must fit within the gripper capacity.
2. **Placement.** Choose the positions in which the chunks are released. For each chunk, also choose which plate opens. At every step, the selected opening side must be free.

The goal is to find a sequence of pick-and-release actions that places all boxes in their target positions while respecting the gripper capacity and avoiding collisions during release.

1.1 Parameters and Derived Constants

The proposed model uses five input parameters. The parameter `n_boxes` denotes the total number of boxes on the pallet. The parameter `boxes_per_row` denotes the number of boxes in each row, and `pallet_rows` denotes the number of rows on the pallet. The parameter `boxes_x_dim` denotes the box dimension parallel to the gripper. Finally, `grip_size` denotes the gripper size S . These parameters are declared as follows:

```
1 par int: n_boxes;  
2 par int: boxes_per_row;  
3 par int: pallet_rows;  
4 par int: boxes_x_dim;  
5 par int: grip_size;
```

From these inputs, the model derives three constants. The first one is the maximum number of boxes that can be picked in a single operation, computed as:

$$\text{max_pkble_boxes} = \left\lfloor \frac{\text{grip_size} + \text{boxes_x_dim}}{\text{boxes_x_dim}} \right\rfloor.$$

This value derives directly from the gripper capacity $S + x$. The second constant is the number of chunks needed to cover one pallet row:

$$\text{chunks_per_row} = \left\lceil \frac{\text{boxes_per_row}}{\text{max_pkble_boxes}} \right\rceil.$$

This is a ceiling division because the last chunk in a row may contain fewer boxes than the maximum allowed number. The third constant is the total number of chunks:

$$\text{n_chunks} = \text{pallet_rows} \cdot \text{chunks_per_row}.$$

It is obtained by multiplying the number of chunks per row by the number of rows. The corresponding MiniZinc declarations are:

```

1 par int: max_pkble_boxes = (grip_size + boxes_x_dim) div boxes_x_dim;
2
3 par int: chunks_per_row = (boxes_per_row + max_pkble_boxes - 1)
4                       div max_pkble_boxes;
5
6 par int: n_chunks = pallet_rows * chunks_per_row;
```

2 Chunking

This part of the model admits several possible encodings. A natural but expensive formulation is a *fully enumerated* encoding, where each chunk explicitly stores the identifiers of all boxes it contains. Since chunks may have different lengths, this representation requires a two-dimensional array with padding values for shorter chunks.

```

1 array[1..n_chunks, 1..max_pkble_boxes] of var 0..n_boxes: chunks;
```

Here, the value 0 is used as a dummy value to represent an empty slot. This representation makes chunk membership explicit and easy to inspect. However, it introduces a large number of decision variables and requires additional constraints to enforce the structure of a valid solution: each box must appear exactly once, and the non-empty positions inside each chunk must be ordered.

A second possibility is a *binary assignment* encoding, where a boolean variable states whether box i belongs to chunk g . This representation can be written as follows:

```

1 array[1..n_chunks, 1..n_boxes] of var bool: belongs_to_chunk;
```

In this case, chunk membership is represented by the truth value of `belongs_to_chunk[g,i]`. Although this encoding is expressive, it is even larger than the fully enumerated representation, and it does not encode contiguity directly: that property must be reconstructed through additional constraints.

For this reason, the adopted model represents each chunk using its *start index* and *length*. An equivalent formulation could use the start and end indices instead. This interval-style representation is more compact because each chunk is described by only two variables, and contiguity is implicit: once the start index and the length are known, the boxes in the chunk are uniquely determined. As a result, the model avoids explicit membership variables and reduces the number of constraints needed to describe valid chunks.

```

1 array[1..n_chunks] of var 1..n_boxes: chunk_start;
2 array[1..n_chunks] of var 1..max_pkble_boxes: chunk_len;

```

The adopted encoding uses two arrays: `chunk_start`, which stores the first box of each chunk, and `chunk_len`, which stores the number of boxes in that chunk.

The following constraints define the position of each chunk in the input stream:

```

1 constraint chunk_start[1] = 1;

```

The first constraint sets the start of the first chunk to box 1. This fixes the beginning of the chunk sequence and ensures that the partition starts from the first box.

```

1 constraint forall(n in 1..n_chunks - 1)(
2   chunk_start[n + 1] = chunk_start[n] + chunk_len[n]
3 );

```

The second constraint makes the chunks consecutive. For each chunk n , the next chunk starts immediately after it, since the start position of chunk $n + 1$ is obtained by adding the length of chunk n to `chunk_start[n]`. This prevents gaps and overlaps between adjacent chunks.

```

1 constraint chunk_start[n_chunks] + chunk_len[n_chunks] - 1 = n_boxes;

```

The final constraint forces the last chunk to end exactly at box `n_boxes`. Together, these constraints ensure that the chunks form a complete partition of the input stream, where every box is covered exactly once.

3 Placement

Once the chunks have been defined, the model must decide where each chunk is placed on the pallet. More precisely, for each chunk it must choose the pallet row on which the chunk is dropped and the horizontal position at which the chunk starts. This is represented by two decision variables:

```

1 array[1..n_chunks] of var 1..pallet_rows: y_row;
2 array[1..n_chunks] of var 1..boxes_per_row: x_start;

```

The variable `y_row[n]` denotes the row assigned to chunk n , while `x_start[n]` denotes the column where that chunk begins.

A first requirement is that every chunk must fit within the horizontal limits of the pallet. Since chunk n starts at column `x_start[n]` and has length `chunk_len[n]`, its last occupied column is `x_start[n] + chunk_len[n] - 1`. This position must not exceed the number of columns available on the pallet:

```

1 constraint forall(n in 1..n_chunks)(
2   x_start[n] + chunk_len[n] - 1 <= boxes_per_row
3 );

```

The second requirement is that no two chunks may occupy the same pallet cell. Geometrically, each chunk g can be seen as an axis-aligned rectangle of height 1: its origin is $(x_start[g], y_row[g])$ and its width is `chunk_len[g]`. The global constraint `diffn(x, y, dx, dy)` enforces non-overlap for a collection of such rectangles, each described by an origin and dimensions. With heights set to 1 for every chunk, it expresses the condition directly:

```
1 constraint diffn(x_start, y_row, chunk_len, [1 | g in 1..n_chunks]);
```

The constraint `diffn(x, y, dx, dy)` enforces that the rectangles defined by origins $(x[i], y[i])$ and dimensions $(dx[i], dy[i])$ do not overlap. In this model, all rectangles have height 1, so `diffn` ensures that chunks assigned to the same row occupy disjoint horizontal intervals.

Together with the chunking constraints, these placement constraints already define a valid pallet layout. The chunking constraints ensure that all `n_boxes` boxes are assigned to chunks, while the placement constraints ensure that those chunks fit inside the pallet and do not overlap.

However, one useful structural property is still only implicit: each pallet row must receive exactly `chunks_per_row` chunks. Nevertheless, leaving this property implicit makes the solver discover it through search.

To make this information explicit, we can add to the model a global cardinality constraint:

```
1 constraint global_cardinality(  
2   y_row,  
3   [r | r in 1..pallet_rows],  
4   [chunks_per_row | r in 1..pallet_rows]  
5 );
```

The constraint `global_cardinality(arr, vals, counts)` enforces that each value `vals[i]` appears in `arr` exactly `counts[i]` times. In this case, every pallet row from 1 to `pallet_rows` must appear exactly `chunks_per_row` times in `y_row`. As a result, the solver knows from the beginning how many chunks must be assigned to each row. Once a row has received the required number of chunks, no further chunks can be placed there, which reduces unnecessary branching during search.

4 Plate Collision

When a chunk is released, the gripper opens one of its two plates. The opening side sweeps the adjacent pallet row: if that row already contains a box overlapping the chunk along x , the plate collides with it. A binary variable encodes the opening side, with 0 meaning the plate opens upward (toward row $r - 1$) and 1 downward (toward row $r + 1$):

```
1 array[1..n_chunks] of var 0..1: plate;
```

An auxiliary variable represents the target row swept by each plate:

```
1 array[1..n_chunks] of var 0..boxes_along_y+1: target;  
2 constraint forall(g in 1..n_chunks)(  
3   target[g] = y_row[g] - 1 + 2*plate[g]  
4 );
```

When `plate[g] = 0` the expression yields `y_row[g] - 1`; when `plate[g] = 1` it yields `y_row[g] + 1`. The domain is widened to $[0, \text{boxes_along_y} + 1]$ so that the boundary values 0 and `boxes_along_y + 1` represent “off the pallet”: they never match a real row index and therefore encode a free edge without any special case.

The no-collision rule then follows. Because release order coincides with belt order, `pred_chunk` is already on the pallet whenever `pred_chunk < current_chunk`. The range `pred_chunk` in

1..`current_chunk`-1 is a compile-time parameter, so the solver generates only the relevant pairs:

```

1 constraint forall(curr_chunk in 2..n_chunks,
2                   pred_chunk in 1..curr_chunk-1)
3 (
4   not(
5     y_row[pred_chunk] = target[curr_chunk]
6     /\ x_start[pred_chunk] < x_start[curr_chunk] + chunk_len[curr_chunk]
7     /\ x_start[curr_chunk] < x_start[pred_chunk] + chunk_len[pred_chunk]
8   )
9 );

```

The body is a direct negation of a single conjunction: `pred_chunk` cannot share the target row of `curr_chunk` while overlapping it along x .

5 Search Strategy

The model adopts a constructive search strategy that exploits a structural property of the problem: the pallet can be filled row by row, top to bottom, and within each row left to right, taking the longest pick the gripper allows at each step. Under this plan, the row above each new chunk is already full when it is placed, so the collision rule naturally forces the plate to open downward into the still-empty rows below, where no collision is possible. The plan is feasible by construction, and a search that follows this order finds a solution with essentially no backtracking.

For each chunk the search must fix four values together: the pallet row `y_row[i]`, the length `chunk_len[i]`, the starting column `x_start[i]`, and the plate direction `plate[i]`. The desired values are, respectively, the smallest available row, the longest pick possible, the leftmost column, and the downward plate.

This intent cannot be expressed directly. The `int_search` annotation visits a single array in index order and applies a single value-selection rule, whereas the strategy above needs four arrays at once and mixed extrema: `indomain_min` on row and column, but `indomain_max` on length and plate. The model works around both limitations with a single construction: it channels the four decisions of each chunk into one `branch` array, laid out in blocks of four, and applies a uniform `indomain_min` to the whole array. For the variables on which a maximum is wanted, the entry stored in `branch` is the *complement*, so that minimizing the entry yields the maximum of the original variable.

```

1 par int: branch_ub = max([boxes_along_y, boxes_along_x, max_pkble_boxes]);
2
3 array [1..4*n_chunks] of var 0..branch_ub: branch;
4
5 constraint forall(i in 1..n_chunks)(
6   branch[4*i-3] = y_row[i] /\
7   branch[4*i-2] = max_pkble_boxes - chunk_len[i] /\
8   branch[4*i-1] = x_start[i] /\
9   branch[4*i]   = 1 - plate[i]
10 );
11
12 solve :: int_search(branch, input_order, indomain_min)
13 satisfy;

```

The `branch` array is laid out so that entries $4i - 3$, $4i - 2$, $4i - 1$, $4i$ correspond to the row, length, column, and plate of chunk i . Visiting the array in `input_order` therefore processes all

four decisions of chunk 1 before moving to chunk 2, and so on.

For row and column, the entries store $y_row[i]$ and $x_start[i]$ directly, so `indomain_min` selects the smallest available values: the topmost row and the leftmost column. For length and plate the desired behavior is the opposite: the search should prefer the longest pick and the downward plate. Since `indomain_min` always minimizes, these two entries store the *complement* of the original variable. The entry `max_pkble_boxes - chunk_len[i]` reaches its minimum of 0 when `chunk_len[i] = max_pkble_boxes`; the entry `1 - plate[i]` reaches its minimum of 0 when `plate[i] = 1`.

Figure 1 shows an example output of the proposed model.

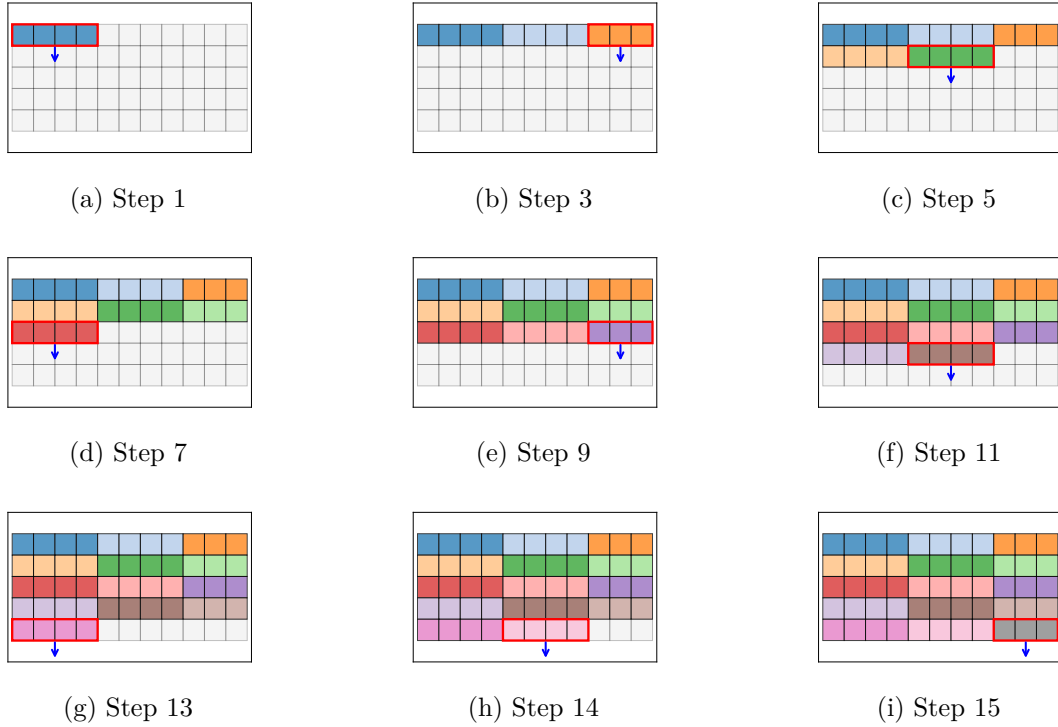


Figure 1: Example of a feasible sequence. The red border marks the chunk being placed at the selected step, while the blue arrow indicates the opening side of the plate.

6 Calling the Model from Python

Python is used to run the MiniZinc model on the benchmark instances. The script loads the model and selects the solver:

```
1 model = Model("model.mzn")
2 solver = Solver.lookup(solver_name)
3 instance = Instance(solver, model)
```

It then sets the instance parameters:

```
1 instance["n_boxes"] = pallet_info["n_boxes"]
2 instance["boxes_x_dim"] = boxes_x_dim
3 instance["boxes_per_row"] = pallet_info["boxes_per_row"]
4 instance["pallet_rows"] = pallet_info["pallet_rows"]
5 instance["grip_size"] = grip_size
```

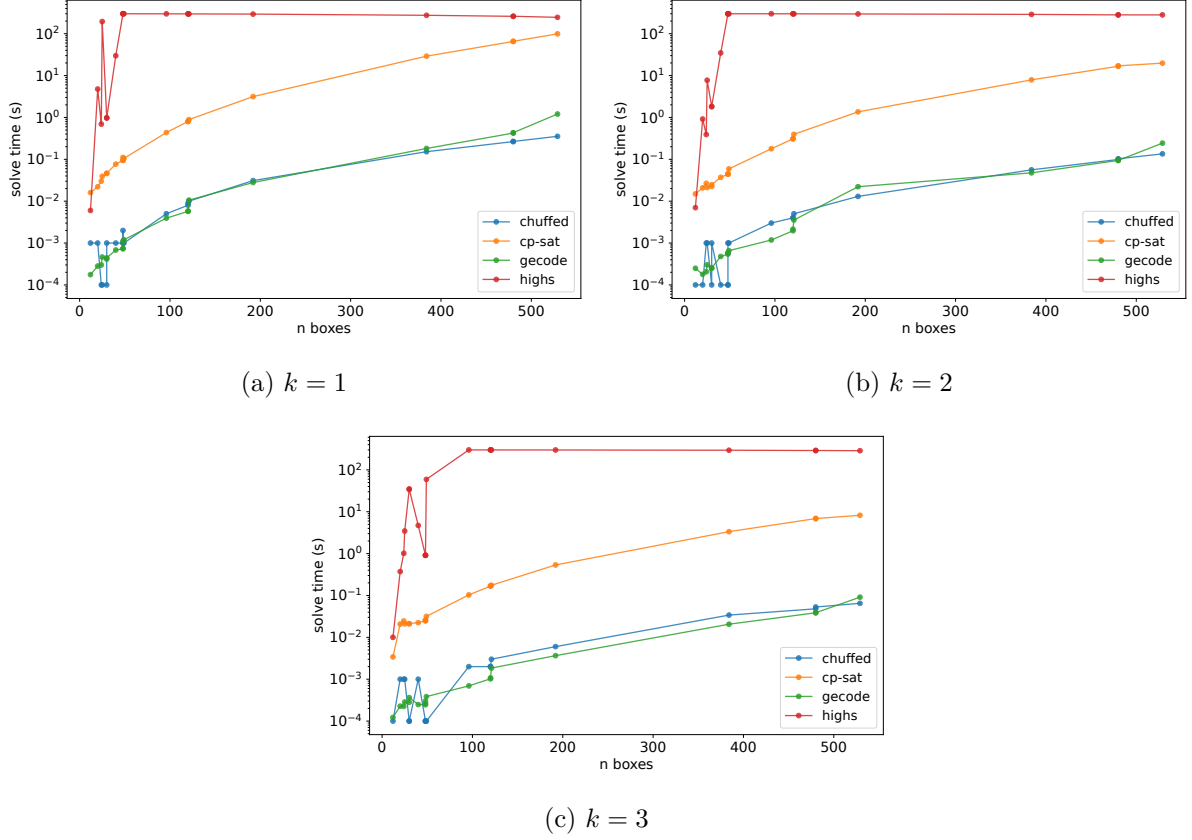


Figure 2: Runtime scaling for the three gripper multipliers $k \in \{1, 2, 3\}$. The y -axis uses a logarithmic scale.

After the parameters are set, the script calls the solver:

```
1 result = instance.solve()
```

The output arrays `chunk_start`, `chunk_len`, `plate`, `y_row`, and `x_start` are then read from the `result` map. They give the grouping of the boxes, the target placement of each group, and the release schedule. Since the belt is FIFO, the release order is given directly by the chunk index.

7 Benchmarking

The benchmark evaluates the MiniZinc model on every (pallet, pack) combination obtained from the pallet presets in Table 1 and the pack sizes in Table 2.

Pallet preset	Dimensions (mm)
Europallet	1200 × 800
Industrie	1200 × 1000
US 40×48	1219 × 1016
Half-pallet	800 × 600
Australian	1165 × 1165

Table 1: Pallet presets used in the benchmark.

Box size (mm)
50 × 50
100 × 100
150 × 150
200 × 200

Table 2: Box sizes used in the benchmark.

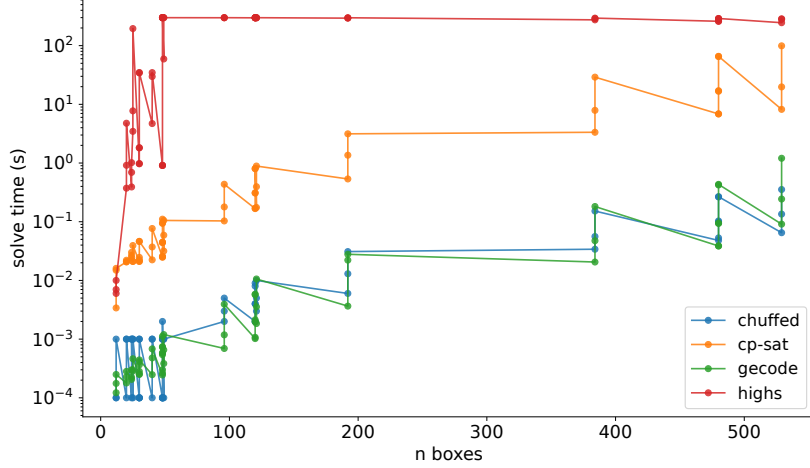


Figure 3: Aggregate runtime scaling over all benchmark instances and solvers. The y -axis uses a logarithmic scale.

For each combination, the model is tested with four solvers: **gecode**, **cp-sat**, **chuffed**, and **highs**. These solvers represent and solve the problem in different ways. **gecode** and **chuffed** mainly rely on constraint propagation, **cp-sat** uses a SAT-based representation, and **highs** uses a linear programming representation. Each run is executed with a timeout of 300 seconds. The benchmark considers three gripper sizes, $S \in \{1, 2, 3\} \cdot \text{boxes_x_dim}$, allowing us to compare how the gripper size affects chunking and releasing performance.

7.1 Scaling by gripper size

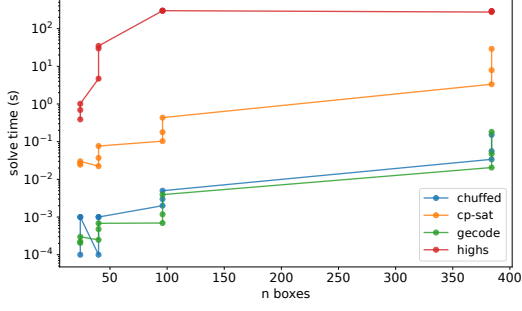
The first comparison fixes the gripper multiplier and measures how the solve time changes with the total number of boxes. Figure 2 shows the resulting scaling curves for $k = 1$, $k = 2$, and $k = 3$. This analysis captures the effect of the gripper size on the proposed model. Larger grippers reduce the number of chunks that must be scheduled. Therefore, the solve time generally decreases as the multiplier increases.

In particular, **gecode** and **chuffed** remain in the lowest runtime range and show the best scaling behaviour. **cp-sat** shows a clearer dependence on the number of boxes, but remains well below the timeout. **highs** is consistently the slowest solver and reaches the timeout range on the larger instances.

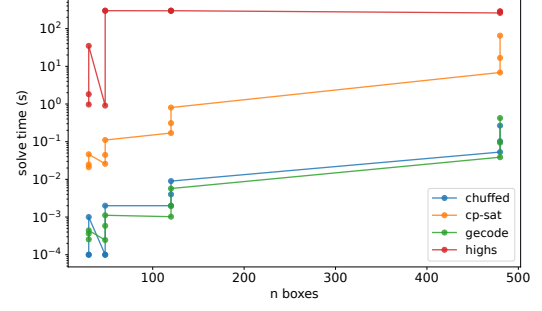
7.2 Overall solver behaviour

After separating the gripper sizes, we aggregate all benchmark runs into a single scaling plot. Figure 3 reports the solve time as a function of the total number of boxes, considering all solvers and all tested instances. This view highlights the global trend across the benchmark and makes the relative behaviour of the solvers easier to compare.

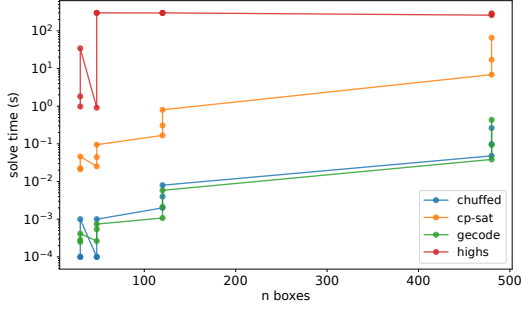
Overall, the comparison confirms that the propagation-based solvers are the most suitable for this model. **gecode** and **chuffed** remain close to the bottom of the plot for most instances, with **gecode** showing slightly more regular behaviour. **chuffed** is also competitive, but its performance is less regular. **cp-sat** shows a stronger dependence on the instance size, although it still remains far from the timeout in most runs. In contrast, **highs** performs worst and several runs approach or reach the timeout.



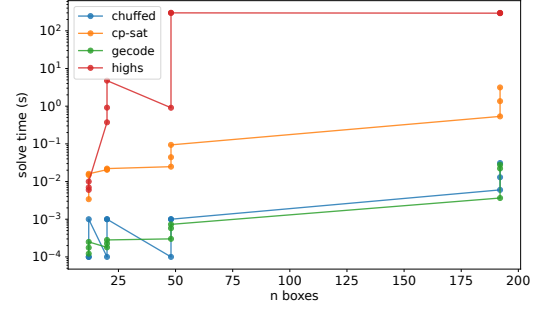
(a) Europallet



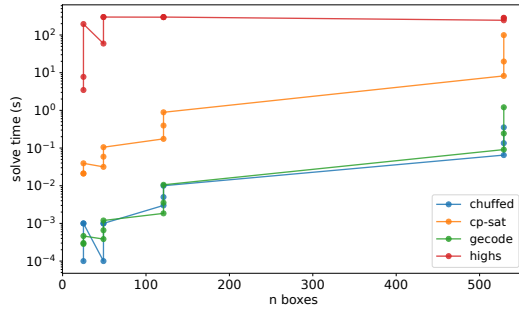
(b) Industrie



(c) US 40×48



(d) Half-pallet



(e) Australian

Figure 4: Runtime comparison across pallet presets. Each subplot aggregates all gripper multipliers for the corresponding preset. The y -axis uses a logarithmic scale.

7.3 Per-pallet breakdown

Another useful comparison studies how the pallet preset influences solver runtime. Differences across presets reflect how `boxes_per_row` and `pallet_rows` determine the number and arrangement of chunks. Figure 4 shows the resulting breakdown.

The relative behaviour of the solvers remains mostly unchanged across pallets: `gecode` and `chuffed` stay close to the bottom of the plots, `cp-sat` is slower but remains well below `highs`, and `highs` is usually the slowest. Larger presets, especially the Industrie, US, and Australian pallets, make the separation between solvers more visible because they include denser instances. The half-pallet instances are smaller, so the differences are less pronounced.

7.4 Flattening and execution overhead

The last analysis compares the solver-reported time with the total runtime measured by the Python benchmark script. This comparison estimates the cost of the steps that occur before the

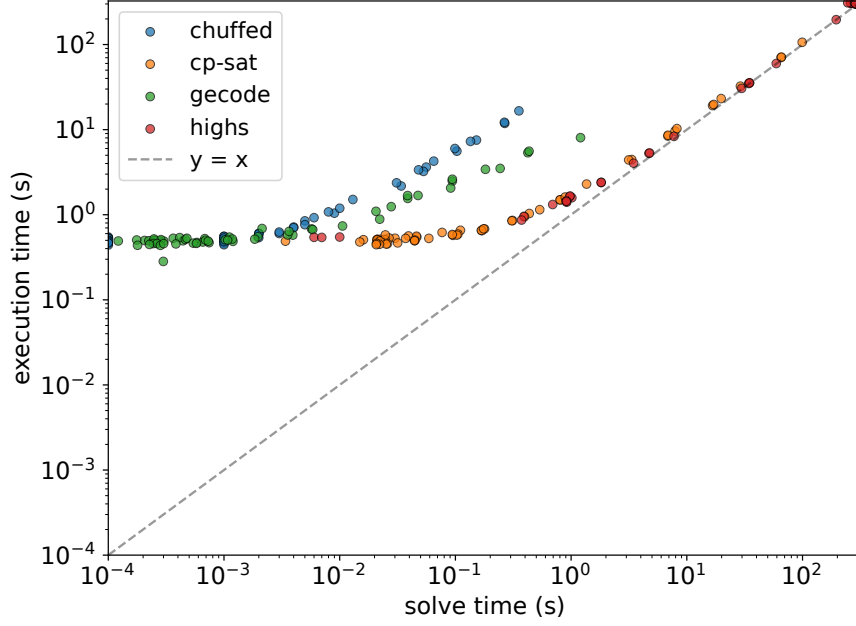


Figure 5: Execution time compared with solver-reported solve time. The diagonal marks equal values; points above it indicate additional overhead before or around the solver call. Both axes use a logarithmic scale.

actual solver search. The main one is MiniZinc flattening: in this phase, the high-level MiniZinc model and the instance data are translated into a lower-level FlatZinc representation, which is then passed to the selected backend solver. The remaining overhead includes Python instance construction, MiniZinc invocation, and process startup.

Figure 5 shows the resulting comparison. The diagonal line marks equal total runtime and solver-reported time. Points close to the diagonal correspond to runs where most of the time is spent inside the solver. Points above the diagonal instead indicate a larger overhead outside the solver; in these cases, flattening and execution costs are more relevant relative to the actual solving time.

The plot shows that this effect is strongest for the fastest runs, especially for **gecode** and several **chuffed** executions. Their solver-reported time is very small, so even a modest overhead produces a visible separation from the diagonal. For slower runs, especially **highs** and the larger **cp-sat** instances, the points are closer to the diagonal. In those cases, the solving phase dominates the total runtime, and the relative overhead becomes less important.

8 Conclusions

The project shows that the robot movement planning problem can be modeled compactly by separating chunking and placement decisions. The interval representation of chunks, based on start index and length, avoids larger membership-based encodings while still capturing the gripper constraints. The release order is then handled through a permutation variable and local neighborhood constraints on the plate direction.

The benchmark shows that the CP-oriented solvers are the most effective on the tested instances. **gecode** is the most stable solver, while **cp-sat** and **chuffed** remain competitive. In contrast, **highs** is consistently slower on this combinatorial formulation. Larger grippers reduce the number of chunks and generally improve runtime, while dense layouts with many small boxes remain the hardest cases.